

Duke University

Grade Automation for Graduate Coursework:

A Product Overview

Harouna Thiam

COMPSCI 391: Independent Study

Prof. Robert Duvall

Dr. Salman Azhar

10 Dec. 2025

This write-up covers the introduction, overview, development, and evaluation of a course assignment auto grader for the purposes of assisting in the evaluation of marks for select assignments of DECISION 616 course Business Computer Applications in the Duke Fuqua School of Business, taught by Dr. Salman Azhar. I think it is impossible to entirely separate discussion of the software tool from discussion of the course and grading protocols, so the entirety of the software tool will be discussed, as well as necessary operational information (course workflows, etc.) from DECISION 616 when appropriate.

EXIGENCE

The primary context for the development of such a product comes all the way back to budget cuts and headcount. Unlike other semesters and courses, particular courses have trouble garnering headcount for instructional Teachers Assistants, often due to issues such as funding cuts. Additionally, particular course structures relay a need for more of a Grading Assistant focus, where assistants primarily grade courses rather than teach instructional content. With enrollment numbers for particular courses at Fuqua exceeding 400+ students, Grading Assistants and particularly administrators have a hard time delegating and optimizing for grading completion. Such a predicament contextualizes a search for some kind of optimization in the grading process. Computer science principles are sought out for such challenges, and in particular in the administrative and grading space, algorithmic innovation can be utilized. To build an ‘algorithm’, one should be able to understand and piece together a **reproducible** and **scalable** set of instructions. In the scope of computer science and software, such a set of instructions should be programmatic and operable by a computer or set of computers. From a

course administrative point of view, one would find the employment of a computer algorithm particularly useful, as it may be able to assist in a number of automated tasks. These relate, more or less, to: the assignment of grades, delegation of TA (Teaching Assistant) tasks, submission filtering, etc. And in the context of very large, primarily assignment oriented, grading focused courses (like DECISION 616), a streamlined and algorithmic approach would handle many such administrative and grading-based grievances, such as naming-convention enforcement, grade penalty enforcement (computing lateness), plagiarism detection, rubric enforcement and evaluation, and much, much more. Such communication brings us to the implementation of a plethora of grading tools, including Gradescope, CodeRunner, DataCamp, Canvas, and others. Each has made waves in solving a particular administrative issue amid automating grading. Gradescope, for example, has built-in test-case evaluating software for grading code samples. Other platforms, like Canvas, a popular LMS (Learning Management Service) used at hundreds of institutions including Duke, have native quiz and assignment functionality which is beneficial for many course and content structures.

SPECIFICITY, PROBLEM SPACE

Duke Fuqua's DECISION 616 course is characterized by many of the factors previously described. Hundreds of students completing the course concurrently, primarily assignment-focused course structure, and relatively low Teaching Assistant count for the student size. Various grading software and LMS' offer tools to assist with such a course structure—not limited to, automated multiple-choice quiz grading, assignment portals for submission, and code evaluation software (supporting multiple languages). However, DECISION 616 offers a unique

challenge. The assignment format is not that of a multiple-choice quiz or code. Rather, each of 400+ students submit Excel assignments. Excel is a desktop software product from the Microsoft Office suite, which offers a **spreadsheet** workbook with **cells**. Each cell may contain some arbitrary value, as well as a specialized sequence of characters called a '**formula**', which may alter the given cell or a range of cells in a multitude of ways. Excel spreadsheets themselves are organized into **workbooks**. Under the hood, Excel spreadsheets aren't simply a document, but a .zip file which contains packets of XML code. Essentially, it is a package of *interlinked XML files* that together determine the structure of the assignment. It is important to note the structure as our tech stack surrounds the utilization of tools that parse these files to find essential information.

In general, none of the regularly available platforms or LMSs are suitable for the widespread Excel auto grading of a large class. There do exist external programs that are able to evaluate Excel files. However, these options (note: ExPrep) are costly and lack cohesion into the professor's specific administrative workflow. Because of these reasons, the development of an in-house Excel course autograder was/is necessary and well-utilized.

STAKEHOLDERS

The development of an autograding software targeting such a specific niche in grading automation has many parties who rely upon it. The specific parties comprise graduate students who will have their work graded partially by the grader, the professor, who greenlights the administrative processes that are endeavored to be automated, and most directly, teaching

assistants who will be using the automated results. From these parties and their various goals/expected outcomes, we derive important **tenets** of our project, or guiding principles that we must abide by:

- 1) **Integrity:** Maintaining fair and non-error grading are essential for student outcomes, who expect their deserved grades.
- 2) **Efficiency:** Ensuring that the automation is effective and streamlined are necessary for the professor's goals of achieving increased grading efficiency.
- 3) **Simplicity:** Prioritizing ease of use (activation and reading produced data) for TAs and other parties.
- 4) **Transparency:** Ensuring that TAs and users can make clear determinations about how the autograder came to a particular score.
- 5) **Reproducibility & Iteration:** *Unsparring* use of repositories, version control, code comments, and documentation for those who may alter the program (professor, future head TAs).

And so, the development of the grader went underway while following each of these five guiding principles. By deriving specific and necessary end goals from each related party, we are able to optimize for a product that can help meet the goals of each involved stakeholder. It is also worth it to reestablish now that the program is designed with regard to the larger grading system. That means that the auto grader will not be acting alone. It is to be integrated into the larger administrative strategy which includes a regular grading period, TA rotation, and grade release period. It is outlined below:

1. Grading assigned to TAs → 2. TAs grade their assigned students → 3. Grades released at the end of the grading period → 4. Repeat

The auto grader is to be utilized in stage (2), where the TAs grade their assigned students.

ASSIGNMENT OVERVIEW

Let us make clear the structure of each assignment. Each **assignment** is composed of many **tasks**, each of which ask the student to complete a particular task like filling in cells, composing Excel formulas, and other basic or complex challenges. Each task will have multiple **steps**. It is at this “step” level that the grader will assign points. The assignment rubrics attribute a certain amount of points for each step, and the grader awards points for the student’s closeness. See the rubric below:

Task/Technique	Possible Points
<u>Task 1: Addressing</u>	
- The title row is centered correctly.	1
- 10 X 10 multiplication table completed with correct values.	1
- All table cells hold formulas.	1
- <i>A single master formula only</i> is entered in the cell at top left, then copied to the rest of the range.	4

Fig. 1

PRODUCT OVERVIEW — DESIGN METHODOLOGY

The ‘autograder’ is quite a vague term. We will elaborate further upon the “non-negotiables”: i.e. the instructor’s (Salman Azhar, Ph.D.) program specifications, what it is this product does, what it attempts to achieve, and how it is integrated into existing administrative and grading workflows. To give more detail, the product developed is a program which outputs grades. According to instructor (Salman Azhar, Ph.D.) specifications, two foundational design specifications are set in stone. The program must:

- 1) ...take in a *single directory file*.
- 2) ...export a *collection of grades*.

According to these specifications, the program developed has a single input and output. Input: single directory which contains the Excel assignments for a particular assignment, and Output: a spreadsheet (.xslm file) which contains the grade breakdown of each student for that particular assignment, as well as “debug” content, which contains the grading logic used by the program. Adhering to the project specifications, all that is necessary is a Python script that is able to search a Desktop for a particular submission file, grab all of the files from within it—and if they’re valid assignments—grade each and generate an .xslm file to export to. To accommodate multiple unique assignments, the grader is composed of multiple files, each for a particular grader. Because of differences in computer architectures, there are two executable files for each assignment. One is a unix executable (for running on Mac) and the other is a .exe file (for running on Windows). Within the program itself, many considerations needed to be made. To

adhere to our strategy for **Reproducibility and Iteration (5)**, which aims to demystify (not that this program is in any way mystic) the program for any engineer who may improve or maintain the program, we use a fundamental programming concept known as *abstraction*. In this, we create multiple ‘modules’ of code to designate a particular programmatic evaluation task to a file. Each file bears the code that programmatically evaluates a particular task. In a ‘main’ file, each module is called on Excel submissions to grade it for all tasks. At a high level, here is the workflow:

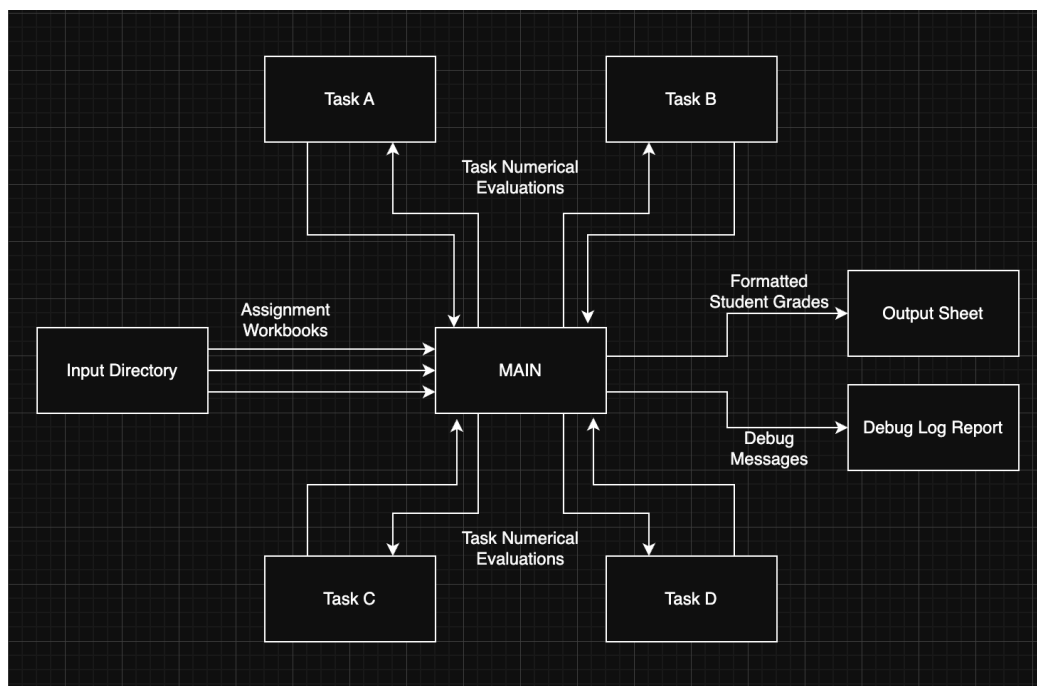


Fig. 2

Where:

- **MAIN** holds function calls to Task A, B, C, D, and any number of individual tasks for that particular assignment. It generates a loop that evaluates each Excel assignment in **INPUT DIRECTORY**. It parses and stores the name of each student, computes their

scores, and passes that data to the **OUTPUT SHEET**. Any errors/messages returned from **TASK** modules are relayed to the **DEBUG LOG REPORT**.

- **TASK** functionality is abstracted away from the logic of **MAIN**. Specific input values are fed to each module, the appropriate evaluations are computed, and a numerical score is returned from the task. If there are any computational errors or a task cannot be programmatically evaluated, the module will return an error message back to **MAIN**.
- **INPUT DIRECTORY** holds the student Excel workbooks. **MAIN** searches the desktop for the correct file, and pulls them.
- **OUTPUT SHEET** is where all grades are exported to.
- **DEBUG LOG REPORT** is a folder containing all of the grading comments for each student.

PRODUCT OVERVIEW — GRADING METHODOLOGY

As mentioned before, there are many benefits of utilizing an *algorithm* in administrative and grading tasks. The lack of human cognitive time/effort overhead and time savings are both large benefits. However, as we are dealing with students, we must place a focus on our design tenets, which include **Integrity (1)**, ensuring that the grades the TA team produces are reliable, accurate, and true to the effort that each student puts forth, as well as accurate to the assignment rubrics. This is relevant notably because the program cannot know or appreciate the spirit of the rubric or assignment—it can merely perform arithmetic on a set of values passed to it. And so, it is extremely important that we are able to correctly interpret the spirit of the rubric as closely as possible and transform those heuristics into code. We'll discuss here the ideas that intertwine the

grading logic of the auto grader with course grading protocols, which include **A) comparison logic** and **B) grading fallbacks** (what to do when the auto grader cannot determine a grade).

A) COMPARISONS: As an important guardrail ensuring **Integrity (1)**, the instructor for this project set a guideline. That is, *evaluations must be made by means of comparison*. What this entails is that by strictly comparing a student submission against a master key, we shrink the probability of our grader marking incorrectly at the cost of missing any other submissions which are technically correct, but do not align 100% with the rubric. What does this mean? Take an example: say the rubric for Task 3 of Assignment 1 dictates that the value “3” must be contained within a particular table. The exact *location* of that table is not specified in the rubric. Say that the master key arbitrarily has that table set starting at cell G6, but a student submission has that table that starts at cell G7. The student’s content is *technically correct*, but by means of strict comparison with the master key, it is marked as wrong since the content is not in the same cells as the master key.

The auto grader generally utilizes this comparison approach. However, to solve the issue of a “fence-post” or “one-off” error explained earlier, Task modules are written to check for values in the specified cell of the master key, as well as the surrounding area (affectionately and aptly named: “*wiggle room*”). That way, students that may have tables “off by one” are marked as correct. There are a few exceptions, though:

- 1) In the case that the rubric specifies that a value **must** be in a particular cell, “wiggle room” is not put into effect.

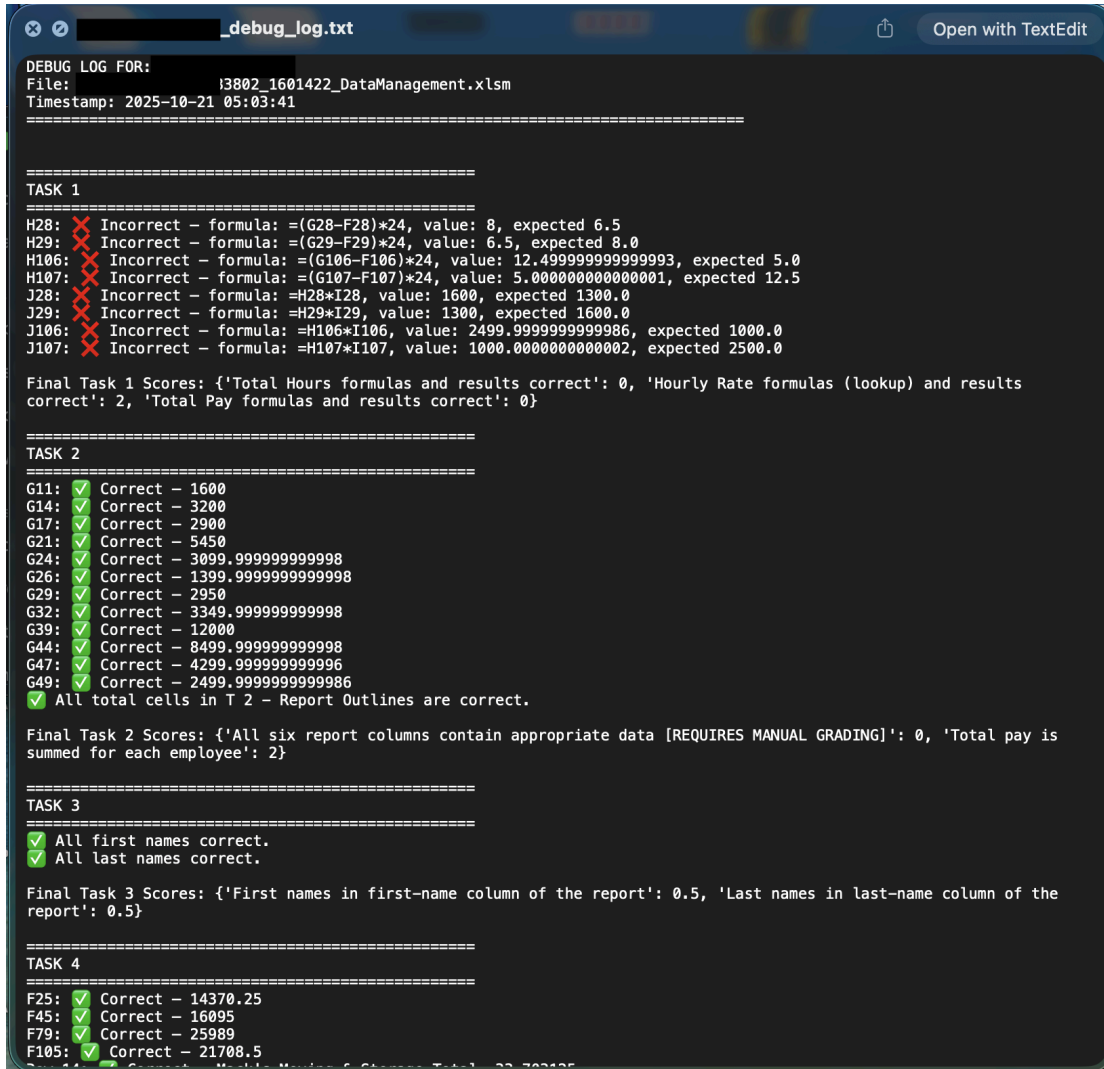
- 2) For tasks involving multi-cell structures such as tables or ranges, the autograder must evaluate these cells collectively rather than individually. The system should allow the entire block to be “wiggled,” or shifted as a unit, to account for minor positional deviations in the student’s spreadsheet. Checking cells independently creates the risk of misattributing correctness, since an identical value may coincidentally appear in an adjacent cell that does not belong to the intended structure. Such cell-level matching would produce logically incorrect grading outcomes. To avoid these false positives, the validator must confirm that the correct configuration of values appears together in the correct relative order, even if the block is offset from the expected origin.

B) GRADING FALLBACKS: To remain in line with our tenet of **Integrity (1)**, any task which cannot be *reliably, scalably, or at all programmatically* evaluated will fall back on manual grading, and TAs are required to check over any 0s that the auto grader awards. Why? Particular Python modules (to be discussed later) such as Openpyxl are built to evaluate Excel cells for particular values, formulas, and other attributes. However, certain issues arise when particular assignments call for checking visual forms of content like graphs. The auto grader is limited in this regard by a particular architecture (Mac x86, ARM); and due to this, some tasks may not be evaluated. If so, this will be notated in the grading output sheet.

PRODUCT OVERVIEW — PROGRAM OUTPUT

As previously discussed, the program output exists in the format of two files: an .xslm file which contains the grades of each student, as well as a directory folder which contains the debug

content of each individual student in the grading session. There is much to be said about the content layout of the file—adhering to our design principle of **Simplicity (3)**, we provide organizational quirks such as: a color code for certain cell values, step-based task breakdown, score totals, and percentage of passed students for a particular task. These features beyond simply the score total exist to aid TAs in filling out each student’s score sheet, which contain a field for the TA to enter. Instead, TAs can utilize the auto grader to fill in a portion or all of those fields. Additionally, the debug output can shed some additional light on what caused the student to lose a point. Since all of the programmatic, cell-checking processes are abstracted away from the TA, the debug logs exist to help them understand why a 0 might be so. The debug logs also serve as a way for TAs to “check” the auto grader’s “work”, abiding by our principle of **Transparency (4)**.



```

DEBUG LOG FOR:
File: i3802_1601422_DataManagement.xlsm
Timestamp: 2025-10-21 05:03:41
=====

TASK 1
=====
H28: ✗ Incorrect - formula: =(G28-F28)*24, value: 8, expected 6.5
H29: ✗ Incorrect - formula: =(G29-F29)*24, value: 6.5, expected 8.0
H106: ✗ Incorrect - formula: =(G106-F106)*24, value: 12.49999999999993, expected 5.0
H107: ✗ Incorrect - formula: =(G107-F107)*24, value: 5.000000000000001, expected 12.5
J28: ✗ Incorrect - formula: =H28*I28, value: 1600, expected 1300.0
J29: ✗ Incorrect - formula: =H29*I29, value: 1300, expected 1600.0
J106: ✗ Incorrect - formula: =H106*I106, value: 2499.999999999986, expected 1000.0
J107: ✗ Incorrect - formula: =H107*I107, value: 1000.000000000002, expected 2500.0

Final Task 1 Scores: {'Total Hours formulas and results correct': 0, 'Hourly Rate formulas (lookup) and results correct': 2, 'Total Pay formulas and results correct': 0}

=====

TASK 2
=====
G11: ✓ Correct - 1600
G14: ✓ Correct - 3200
G17: ✓ Correct - 2900
G21: ✓ Correct - 5450
G24: ✓ Correct - 3099.999999999998
G26: ✓ Correct - 1399.999999999998
G29: ✓ Correct - 2950
G32: ✓ Correct - 3349.999999999998
G39: ✓ Correct - 12000
G44: ✓ Correct - 8499.999999999998
G47: ✓ Correct - 4299.999999999996
G49: ✓ Correct - 2499.999999999986
✓ All total cells in T 2 - Report Outlines are correct.

Final Task 2 Scores: {'All six report columns contain appropriate data [REQUIRES MANUAL GRADING]': 0, 'Total pay is summed for each employee': 2}

=====

TASK 3
=====
✓ All first names correct.
✓ All last names correct.

Final Task 3 Scores: {'First names in first-name column of the report': 0.5, 'Last names in last-name column of the report': 0.5}

=====

TASK 4
=====
F25: ✓ Correct - 14370.25
F45: ✓ Correct - 16095
F79: ✓ Correct - 25989
F105: ✓ Correct - 21708.5

```

Fig. 3

TECH STACK & VERSION CONTROL

The autograder was developed using Python as the primary programming language due to its extensive ecosystem of data-processing libraries, strong community support, and suitability for automation tasks. The core functionality of the system relies on one major component of the Python ecosystem: Openpyxl, which makes up the full extent of Excel ingestion, validation, and

output formatting. The main dependency of the project is Openpyxl, a Python library designed for reading, parsing, and manipulating Excel workbooks in the Office Open XML (.xlsx) format. Openpyxl provides low-level access to spreadsheet structures, including cell objects, workbook and worksheet relationships, chart objects, styles, fills, borders, and color. Openpyxl's function allows the program to dive into Excel's underlying XML schema, making it possible to traverse cell-level metadata and access the necessary student data—which are essential to all of the grading tasks that the auto grader supports.

The project was developed under a Git-based version control workflow, hosted through a private repository to ensure academic integrity. Version control played a vital role in maintaining the stability of the autograder across multiple iterations and assignments, although there was an issue with the loss of one grading module due to a computer wipe and lack of version control for that particular grading module. I learned a strong lesson, that it is important to hit 'git commit' after every work session. In this, the grader adheres to **Reproducibility and Iteration (5)**, which makes it a goal to utilize version control to make technical maintenance of the program easier.

UTILIZATION & FEEDBACK

An email request for feedback was sent to TAs who used the program following the last weeks of the semester. TAs were asked a collection of brief questions:

- How was your experience using the auto-graded results?

- Did the auto-graded results save (compared to grading assignments without it) a **1)** *significant amount of time*, **2)** *non-significant amount of time*, **3)** *no time at all*, or **4)** *did it take longer than just grading normally*?
- Rate the time lost/saved on a scale from **0** to **10**, where 0 is "My work took exceptionally longer using the grader", 10 is "My time was tremendously saved", and 5 is "no time lost or saved".
- How easy or difficult was it to find specific names/grades for students on the spreadsheet? Did color-coding help or hurt your progress?
- Did you encounter any technical or scoring issues with the auto-grader? If so, please explain briefly. If not, write "N/a".
- In a few words, describe how you feel about the auto grader. What would you improve?

User feedback is extremely important for a program. By receiving user feedback, one is able to understand the pain points of their application, revise, iterate, and make the product better than what it was. By asking these specific questions, I sought to understand the effectiveness and utilization of our design tenets. Specifically, did the grader increase efficiency, prioritize ease of use for TAs, and ensure that TAs and users can make clear determinations about how the autograder came to a particular score (Efficiency (2), Simplicity (3), Transparency (4))? Below are my findings on helpfulness, including **A)** Navigation Efficiency, **B)** Visual Design, and **C)** Technical Challenges.

Their responses gave a mixture of positive sentiment, early-stage adjustment challenges, and clear indications of where Version 2 of the autograder should focus improvement efforts. Most

TAs reported that the autograder was generally useful during the grading process. Several noted that the tool became easier to use over time as they became more familiar with its structure. One TA described the experience as “good and helpful.” Another noted that it was “fair” but became better after they gained familiarity. One initially found it difficult and chose not to use it at first, but later acknowledged the tool’s value after revisiting it. A recurring theme was that there was an initial learning curve where learning how to efficiently navigate a new spreadsheet was difficult, especially during a busy academic term.

A) **GRADING EFFICIENCY:** Feedback on efficiency varied, as some TAs reported that the autograder saved a significant amount of time. Others reported non-significant time savings, largely because they double-checked the autograder's results during early use. One TA found usage easier over time but noted that initial unfamiliarity reduced time savings. Only one TA rated time savings below the average (4/10), primarily due to workflow friction (scrolling between graded vs. ungraded portions). Reported time-savings scores ranged from 4 to ~7, with most around 6. No TA reported that the tool increased time burden overall. A few TAs noted that distrust of automation early in the semester led them to manually re-check student work, which reduced efficiency until familiarity increased.

B) **VISUAL DESIGN:** Most TAs found color-coding helpful. It supported organization, reduced visual clutter, and helped them quickly locate the parts they were responsible for. They also found name lookup moderately easy, though one TA noted difficulty because

names in the autograder results did not match the Canvas ordering, requiring manual cross-referencing. Some suggested separating first and last names for improved readability. Overall, the visual design and formatting were seen as strengths.

C) TECHNICAL CHALLENGES: Some TAs encountered minor technical or scoring problems. Common issues were False negatives; like when students placed valid answers in a *different but reasonable cell*, the autograder marked the answer wrong. To note, this was an intentional design, as it was a required aspect of maintaining grading **Integrity (1)**. For any 0s, TAs should manually check. However, no TAs reported outright incorrect scoring that misled their final grading decisions.

TAs generally expressed positive or cautiously positive impressions. Some words used were “Helpful”, “Good”, “Pretty helpful and easy on the eye”, as it became more useful over time for many TAs. In terms of achieving **Efficiency (2)**, Most TAs reported meaningful though sometimes limited time savings, with scores averaging around 6/10. Efficiency improved as familiarity increased. The largest efficiency losses came from workflow friction (Canvas ordering mismatch) and partial grading coverage. For **Simplicity (3)**, color-coding and the visual layout supported ease of use. Difficulties stemmed primarily from name-order mismatches and the mental overhead of learning a new tool mid-semester. For **Transparency (4)**, while most TAs trusted the tool, lack of information on cell-specific matching and uncertainty about what was or wasn’t graded reduced transparency. TAs want clearer delineation of grading boundaries and more flexible evaluation logic.

CONCLUSION & NEXT STEPS

The feedback collected throughout the semester makes clear that the autograder successfully addressed several core administrative challenges in DECISION 616, while also revealing important areas for refinement. The tool seemingly improved grading output, standardized evaluation, and provided a structured, reproducible method for scoring complex Excel-based assignments. However, as with any software deployed in a real instructional environment, Version 1 faced many limitations which could be fixed in a second version. In direct response to TA feedback—and guided by the project’s principles of Efficiency, Simplicity, Transparency, Integrity, and Reproducibility—the next version would benefit from expanded grading coverage, more flexible comparison logic, improved name matching and navigation, and streamlined debugging output.

Because of reduced TA headcount and high enrollment in particular courses—such a tool fills an important gap in automated grading and administration. With the project’s foundational design tenets derived from analyzing stakeholder goals, it respects the integrity of student work, supports graders, and maintains the high bar of grading for students. The autograder has already delivered measurable benefits to the DECISION 616 team. TAs reported moderate to significant time savings, clearer organization, and improved consistency across the grading process. At the same time, their feedback revealed opportunities to strengthen the tool’s accuracy, usability, and transparency. In the end, this project also goes back to the broader goal of software development: creating tools that not only work, but work well for people. With continued refinement, the

program has the potential to become an invaluable asset not only for DECISION 616, but for a broader range of courses that rely on spreadsheet-based analytics.